Solution: Running Sum of 1d Array (easy)

Problem Statement

Given a one-dimensional array of integers, create a new array that represents the running sum of the original array.

The running sum at position `i` in the new array is calculated as the sum of all the numbers in the original array from the 0th index up to the `i`-th index (inclusive). Formally, the resulting array should be computed as follows: `result[i] = sum(nums[0] + nums[1] + ... + nums[i])` for each `i` from 0 to the length of the array minus one.

Examples

Example 1

- **Input**: `[2, 3, 5, 1, 6]`
- **Expected Output**: `[2, 5, 10, 11, 17]`
- **Justification**:
  - For i=0: 2
  - For i=1: 2 + 3 = 5
  - For i=2: 2 + 3 + 5 = 10
  - For i=3: 2 + 3 + 5 + 1 = 11
  - For i=4: 2 + 3 + 5 + 1 + 6 = 17

Example 2

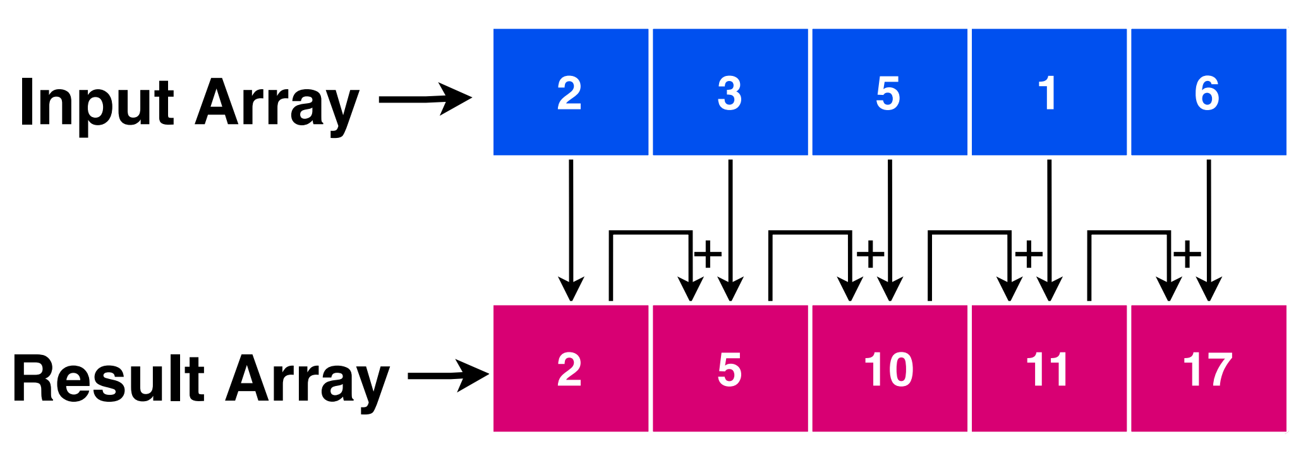
- **Input**: `[1, 1, 1, 1, 1]`
- **Expected Output**: `[1, 2, 3, 4, 5]`
- **Justification**: Each element is simply the sum of all preceding elements plus the current element.

Example 3

- **Input**: `[-1, 2, -3, 4, -5]`
- **Expected Output**: `[-1, 1, -2, 2, -3]`
- **Justification**: Negative numbers are also summed up in the same manner as positive ones.

Solution

To find a solution of this problem, we can employ a straightforward approach. Starting from the first element of the input array, we can traverse through each element, cumulatively summing up the values as we proceed and placing the running total at the corresponding index in the resulting array.



Initially, the first element of the output array is the same as the input array since there are no preceding elements to add. From the second element onwards, **every element in the output array is the sum of the current input element and the previous element in the output array**. This is because the previous element in the output array already contains the cumulative sum of all the previous elements in the input array.

Here is the algorithm broken down into steps:

1. **Initialize a Variable for Running Sum**: Start by setting a variable (let's call it `runningSum`) to 0. This variable will keep track of the cumulative sum as you iterate through the array.
2. **Iterate Through the Array**: Loop through each element of the array. You can use a standard `for` loop for this purpose.
3. **Update Running Sum**: In each iteration, add the current element's value to `runningSum`. This step updates the running total with the value of the current element.
4. **Replace Current Element**: After updating the `runningSum`, replace the current element in the array with the value of `runningSum`. This effectively changes each element to the sum of all preceding elements including itself.
5. **Continue Until the End of the Array**: Continue steps 3 and 4 for each element in the array. By the end of the loop, each array element will have been replaced with the cumulative sum up to that point.
6. **Return the Modified Array**: After completing the iteration through the array, return the modified array. This array now contains the running sums instead of the original values.

This approach guarantees that we traverse through the input array only once, summing up the elements as we go along and avoiding recalculating any previously computed sums. Thus, it is computationally efficient while also being memory-efficient, as we don't store intermediate results in additional data structures.

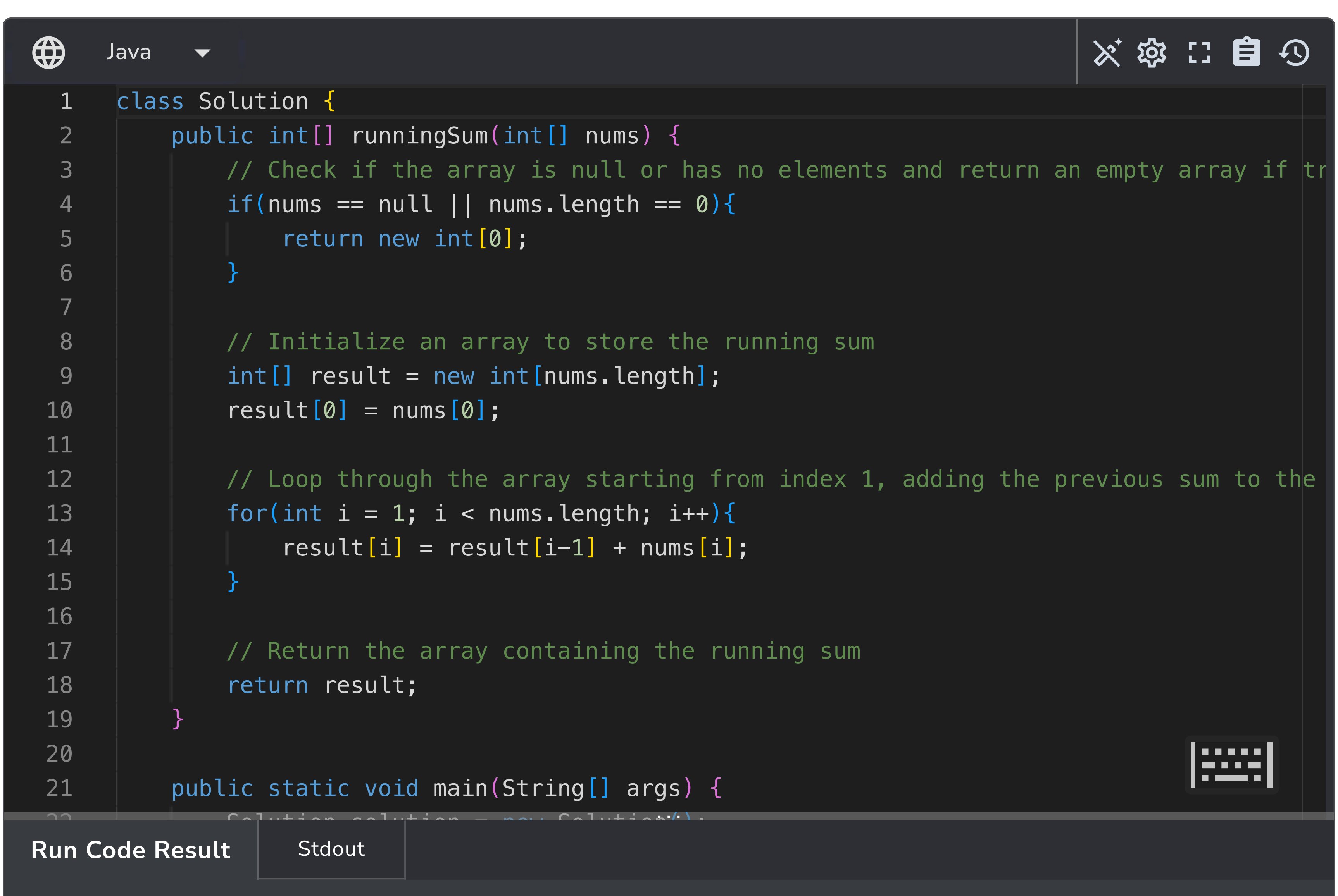
Algorithm Walkthrough

Let's consider an example input array `[2, 3, 5, 1, 6]` to understand how the algorithm works:

- Initialize the resulting array with the same length as the input array. Let's say `result[]`.
- Set `result[0]` equal to `nums[0]` since the first element does not have any previous elements to add. So, `result = [2, _, _, _, _]`
- For `i` from 1 to `len(nums)-1`:
  - Set `result[i] = result[i-1] + nums[i]`.
  - For i=1: `result[1] = result[0] + nums[1] = 2 + 3 = 5` so `result = [2, 5, _, _, _]`
  - For i=2: `result[2] = result[1] + nums[2] = 5 + 5 = 10` so `result = [2, 5, 10, _, _]`
  - For i=3: `result[3] = result[2] + nums[3] = 10 + 1 = 11` so `result = [2, 5, 10, 11, _]`
  - For i=4: `result[4] = result[3] + nums[4] = 11 + 6 = 17` so `result = [2, 5, 10, 11, 17]`
- Return the result array `[2, 5, 10, 11, 17]`

Code

Here is the code for this algorithm:



```java
class Solution {
    public int[] runningSum(int[] nums) {
        // Check if the array is null or has no elements and return an empty array if tr
        if(nums == null || nums.length == 0){
            return new int[0];
        }

        // Initialize an array to store the running sum
        int[] result = new int[nums.length];
        result[0] = nums[0];

        // Loop through the array starting from index 1, adding the previous sum to the
        for(int i = 1; i < nums.length; i++){
            result[i] = result[i-1] + nums[i];
        }

        // Return the array containing the running sum
        return result;
    }

    public static void main(String[] args) {
```

You must run your code first

Time Complexity:

The algorithm essentially involves a single loop that iterates through the input array to compute the running sum. Here's the breakdown:

- **O(n)**: We iterate through all (n) elements of the input array exactly once. Inside the loop, we perform a constant amount of work (a sum and an assignment). Hence, the time complexity is linear.

Space Complexity:

- **O(1)**: The algorithm uses a constant amount of extra space. The result array does not count towards the space complexity since it's the expected output. However, no additional data structures grow with the input size, meaning that the algorithm uses a constant amount of additional memory.

In summary:

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

The algorithm is quite efficient, as it computes the running sum in a single pass through the input and does not use additional memory that grows with input size. This means it can handle large input arrays effectively, provided they can be stored in memory.

← Previous
Next →
Problem 1: Running Sum Of 1d Array (Easy)
Problem 2: Contains Duplicate (Easy)
☐ Mark as Completed